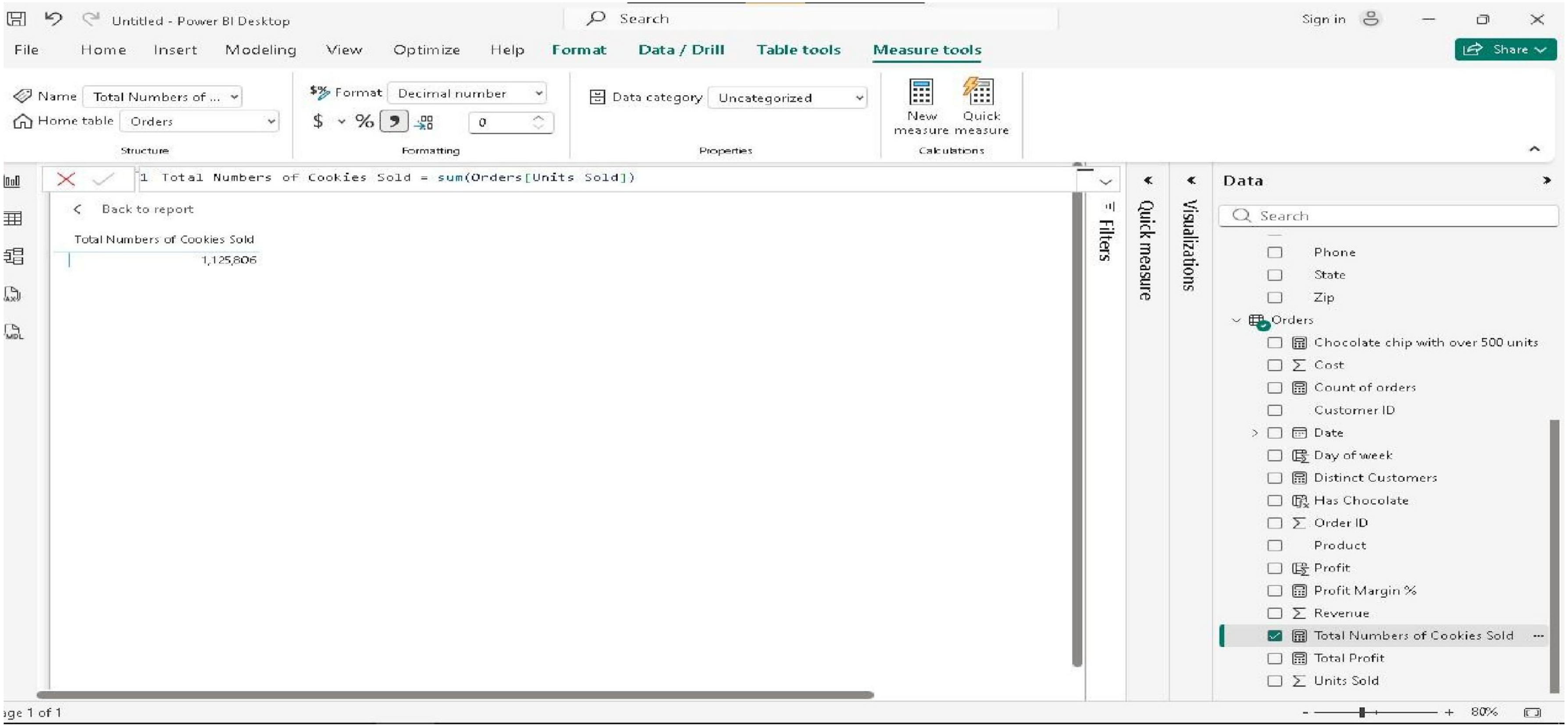
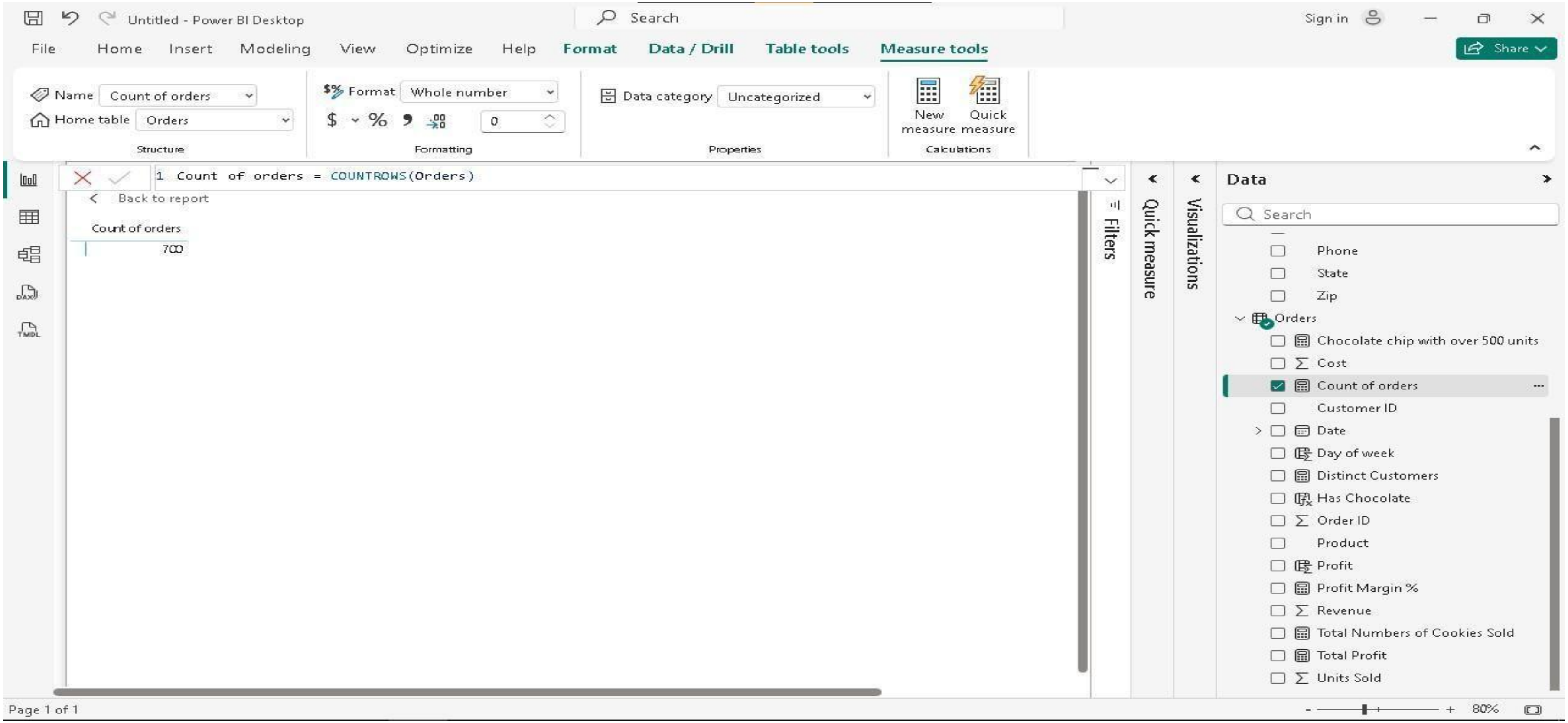
Applying DAX Fundamentals in Power BI: From Measures to Contextual Analysis

Neil Bryant O. Galindez
Instructor: Joseph A. Vistal
IS 107 - Business Intelligence

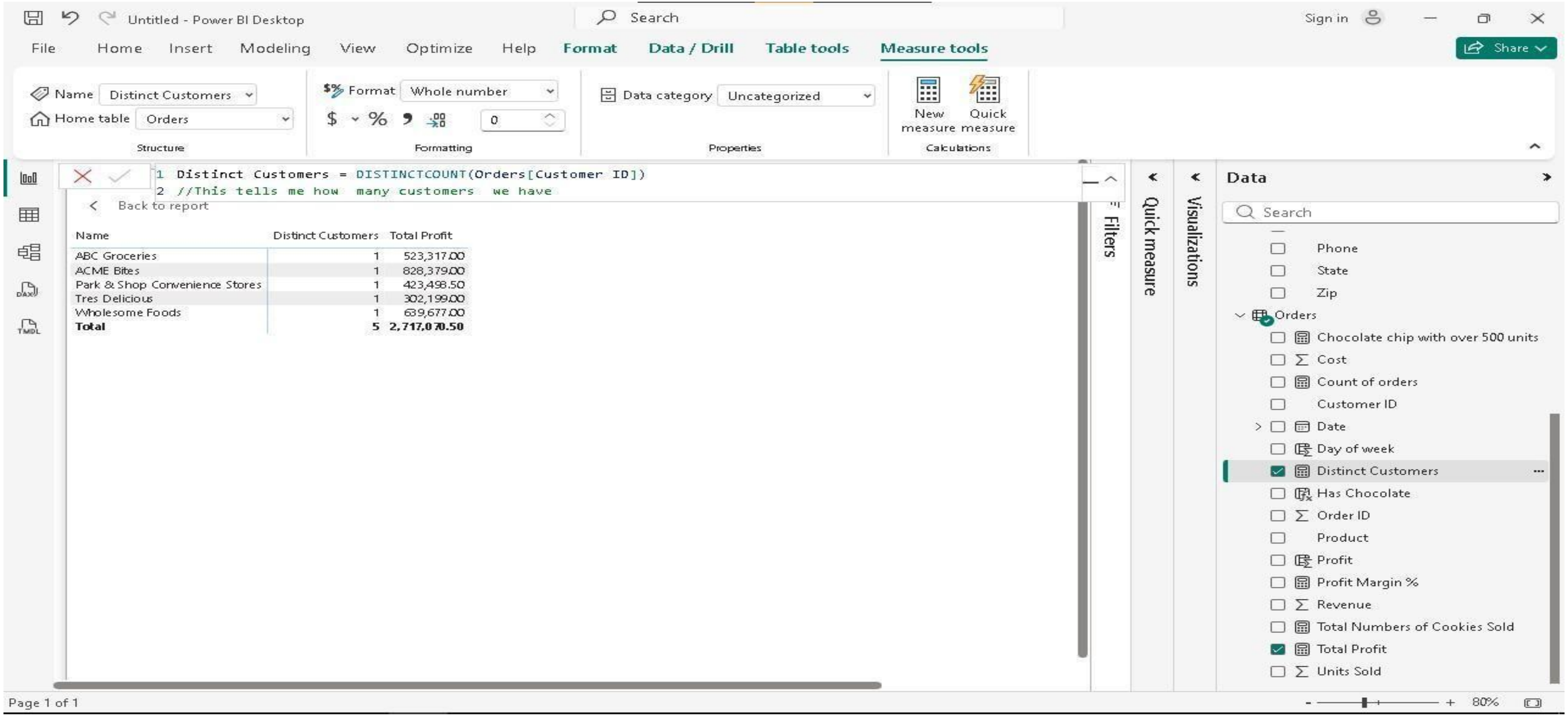
Formula:Total Numbers of Cookies Sold = sum(Orders[Units Sold])



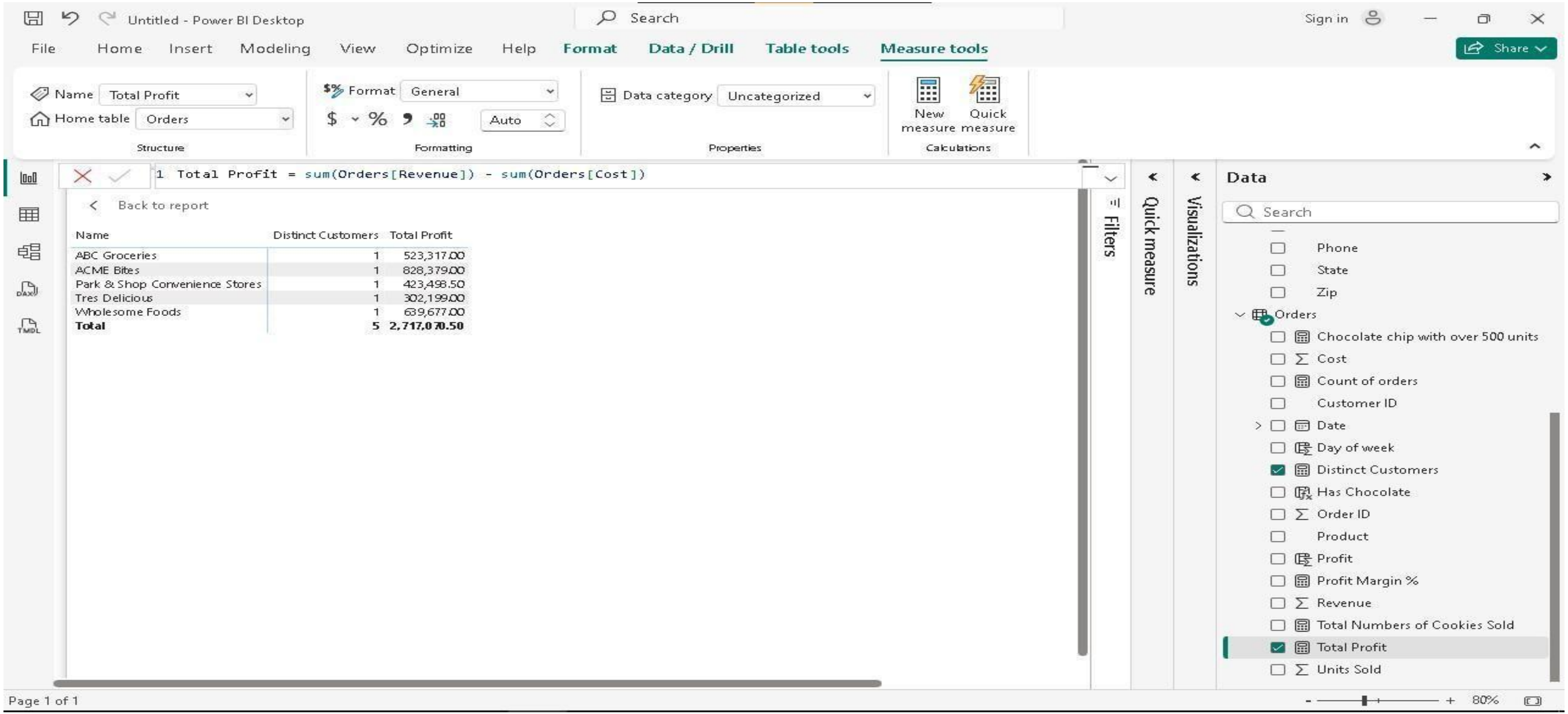
Formula: Count of orders = COUNTROWS(Orders)



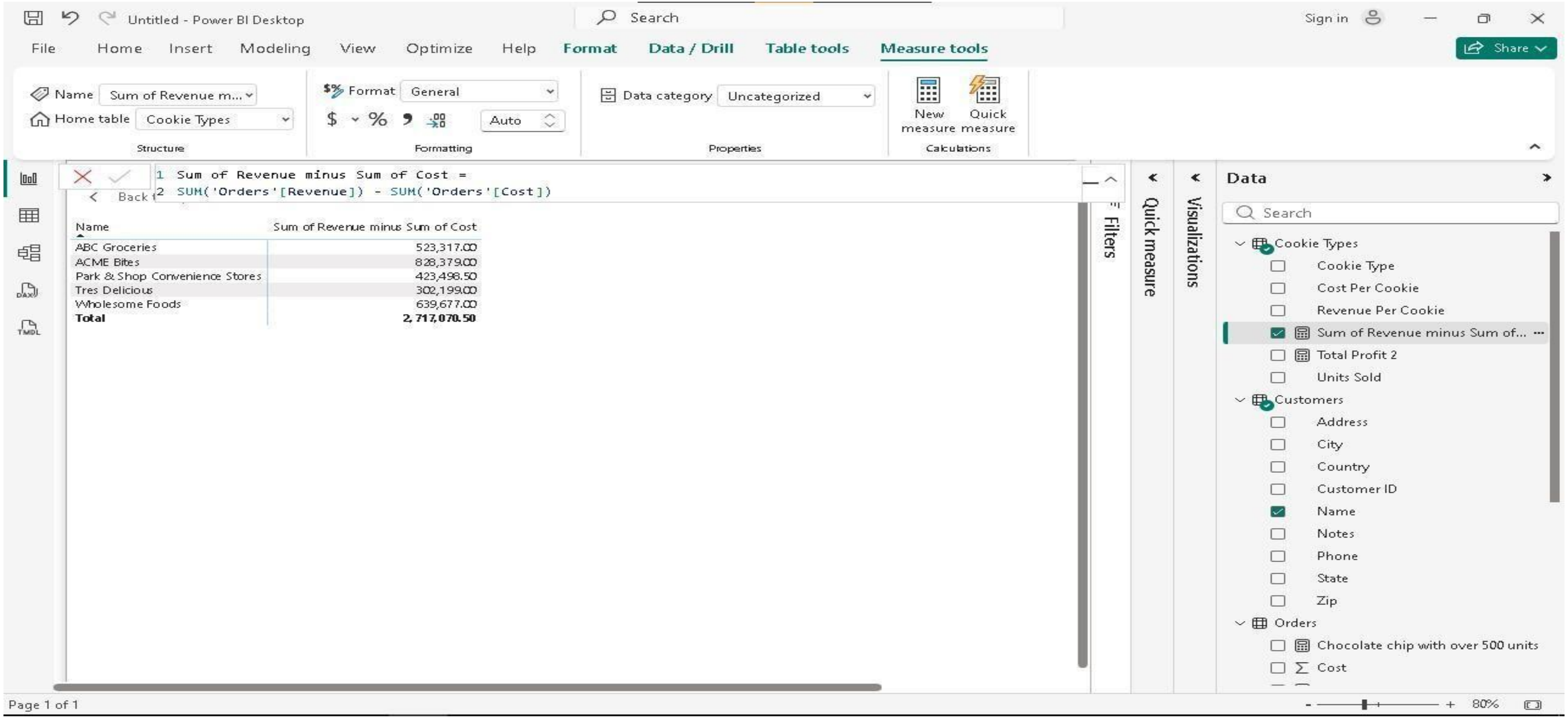
Formula:Distinct Customers = DISTINCTCOUNT(Orders[Customer ID]) //This tells me how many customers we have



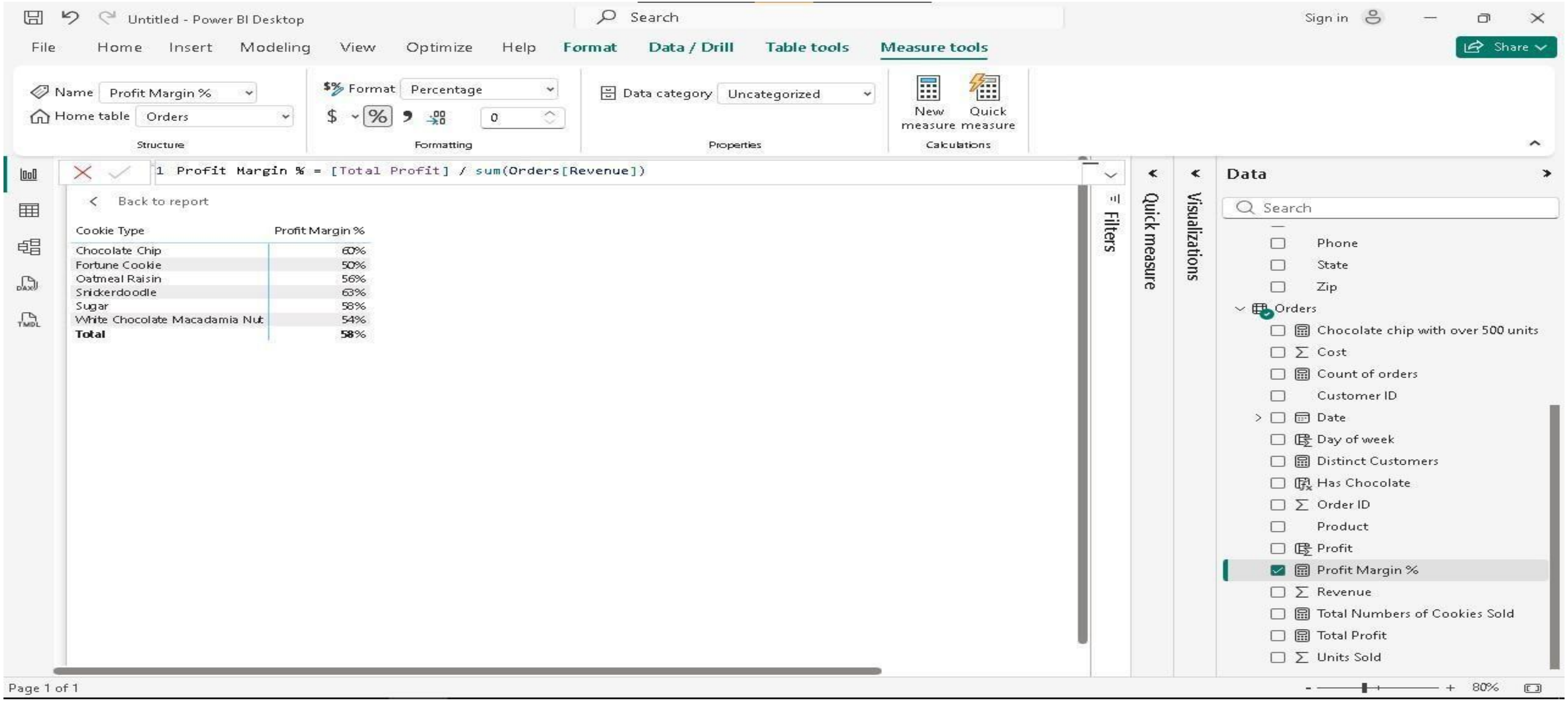
Formula: Total Profit = sum(Orders[Revenue]) - sum(Orders[Cost])



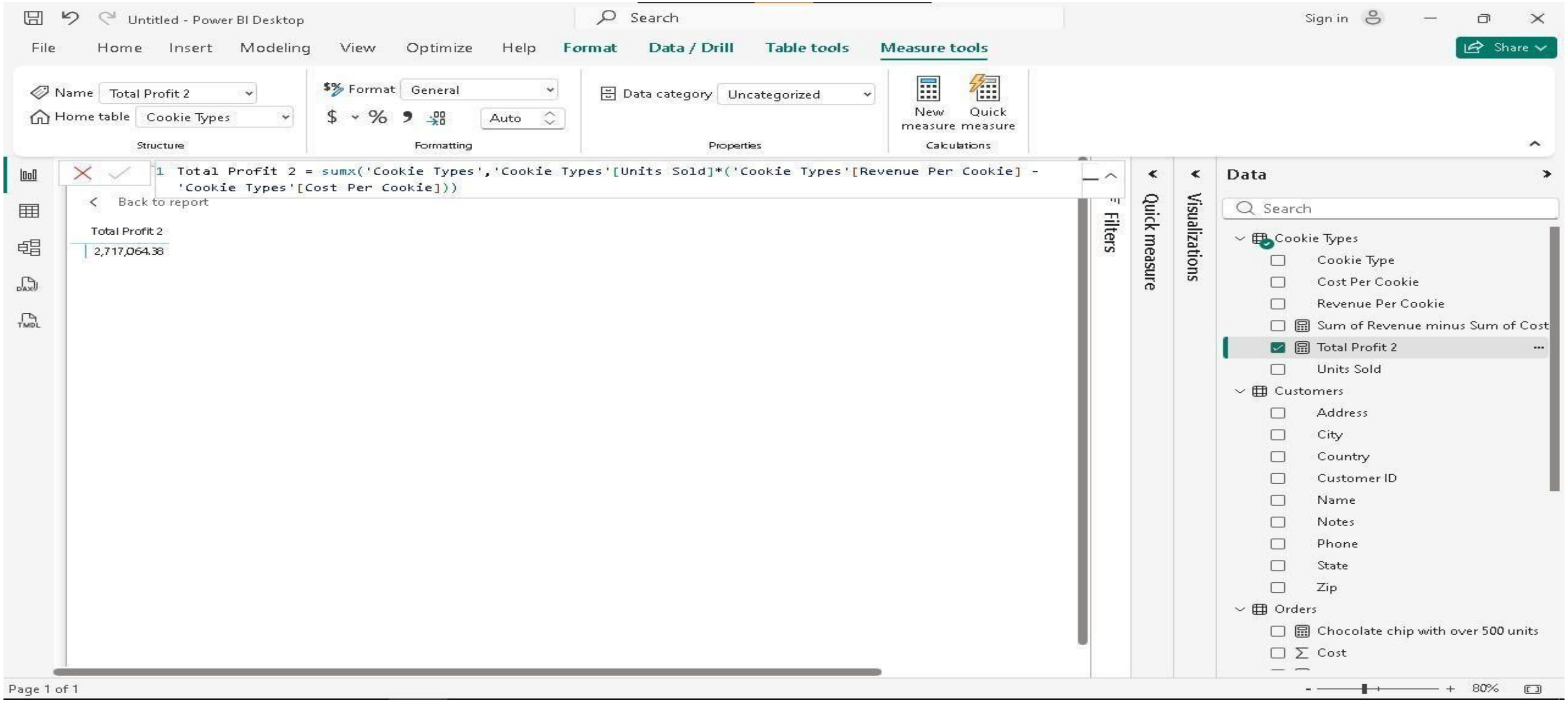
Formula: Sum of Revenue minus Sum of Cost = SUM('Orders'[Revenue]) - SUM('Orders'[Cost])



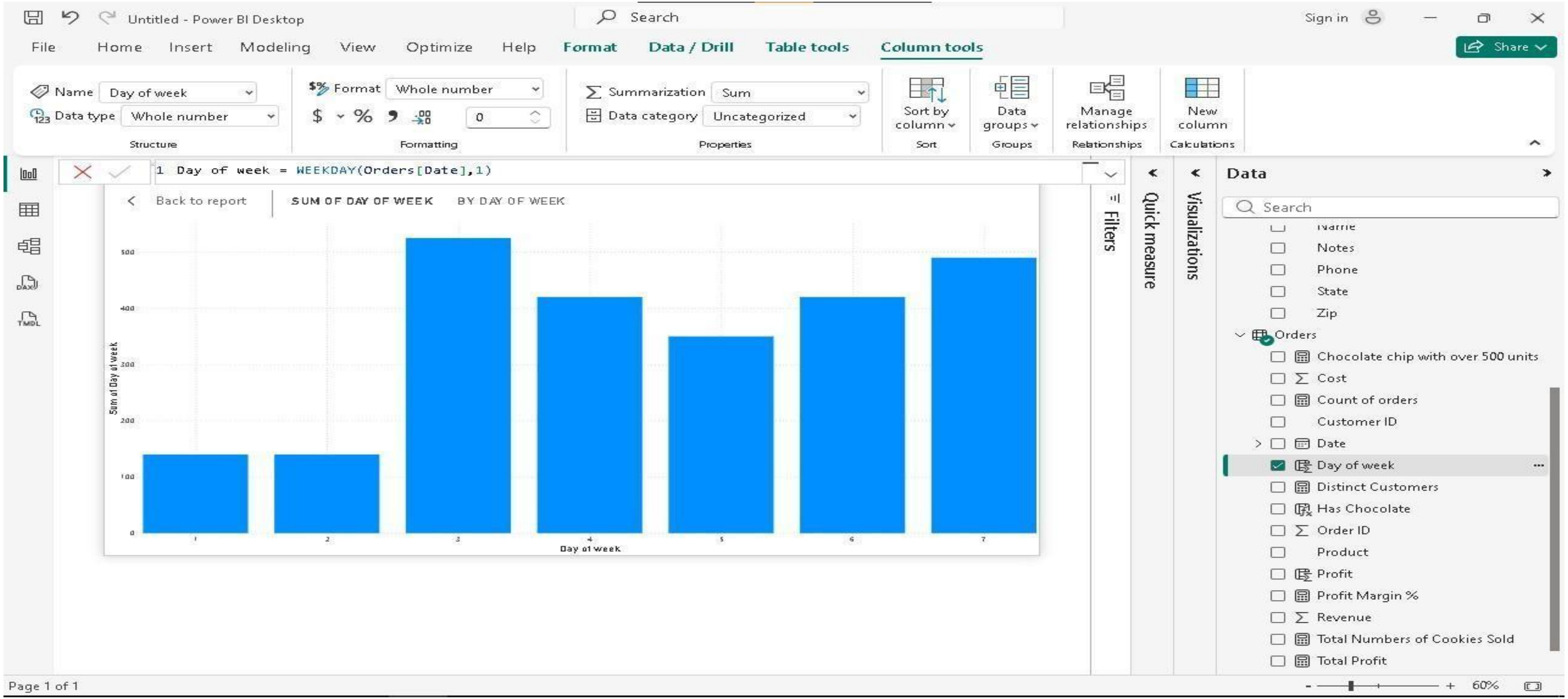
Formula: Profit Margin % = [Total Profit] / sum(Orders[Revenue])



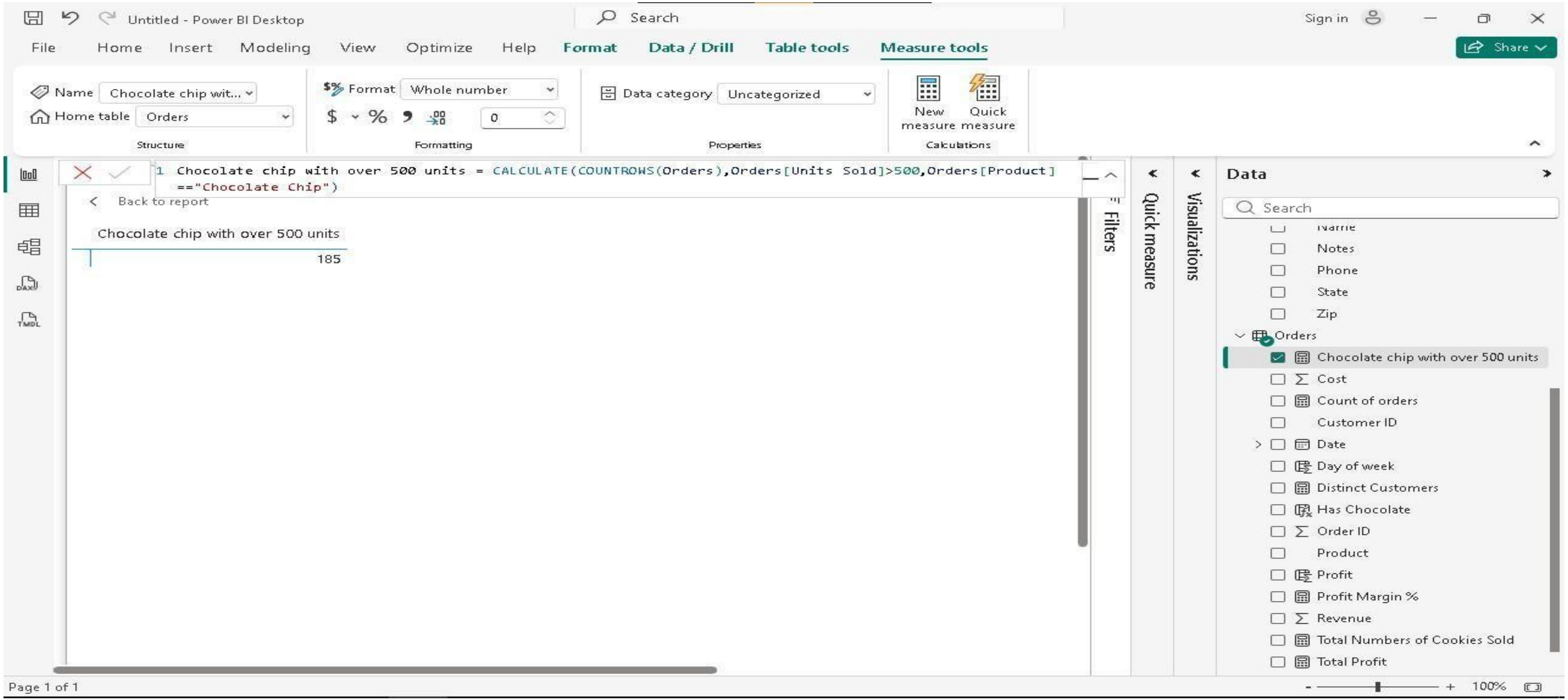
Formula: Total Profit 2 = sumx('Cookie Types','Cookie Types'[Units Sold]*('Cookie Types'[Revenue Per Cookie] - 'Cookie Types'[Cost Per Cookie]))



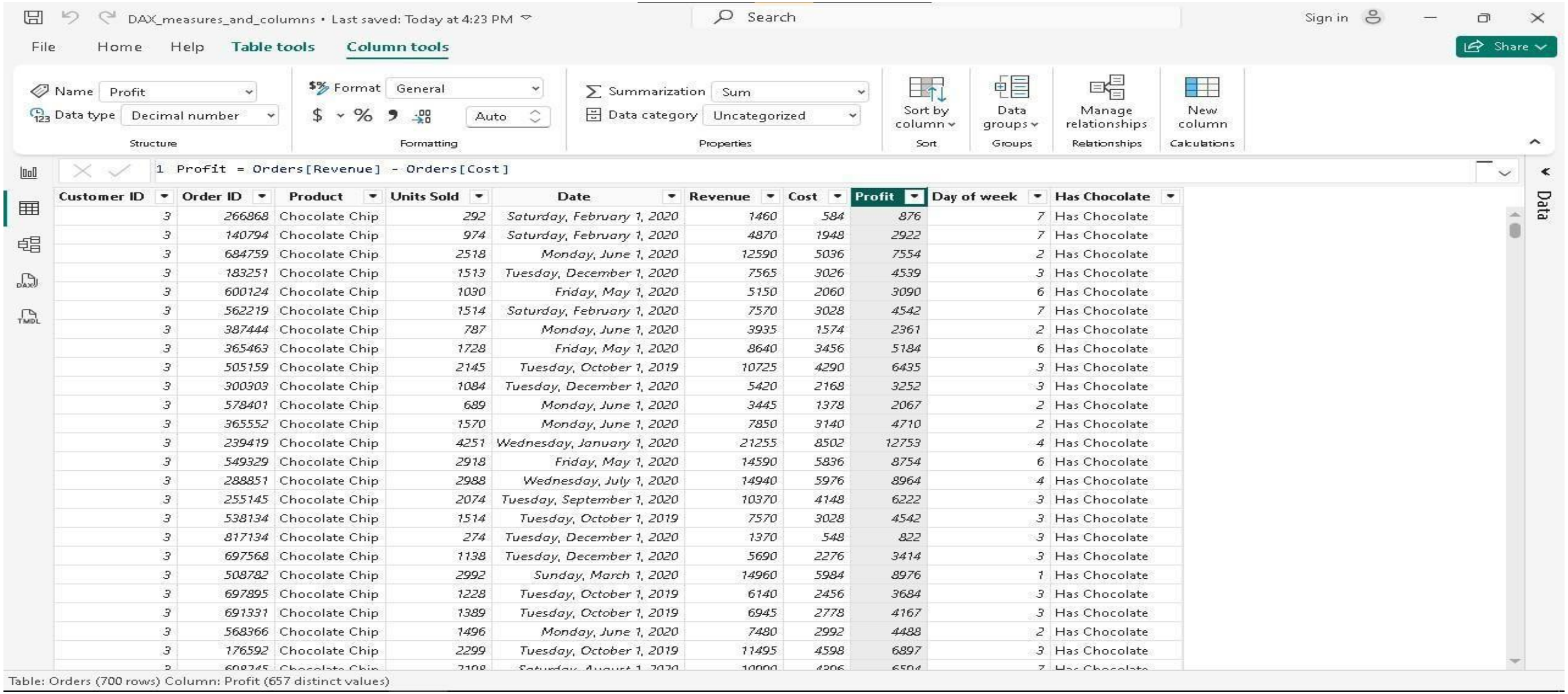
Formula: Day of week = WEEKDAY(Orders[Date],1)



Formula: Chocolate chip with over 500 units = CALCULATE(COUNTROWS(Orders),Orders[Units Sold]>500,Orders[Product]=="Chocolate Chip")

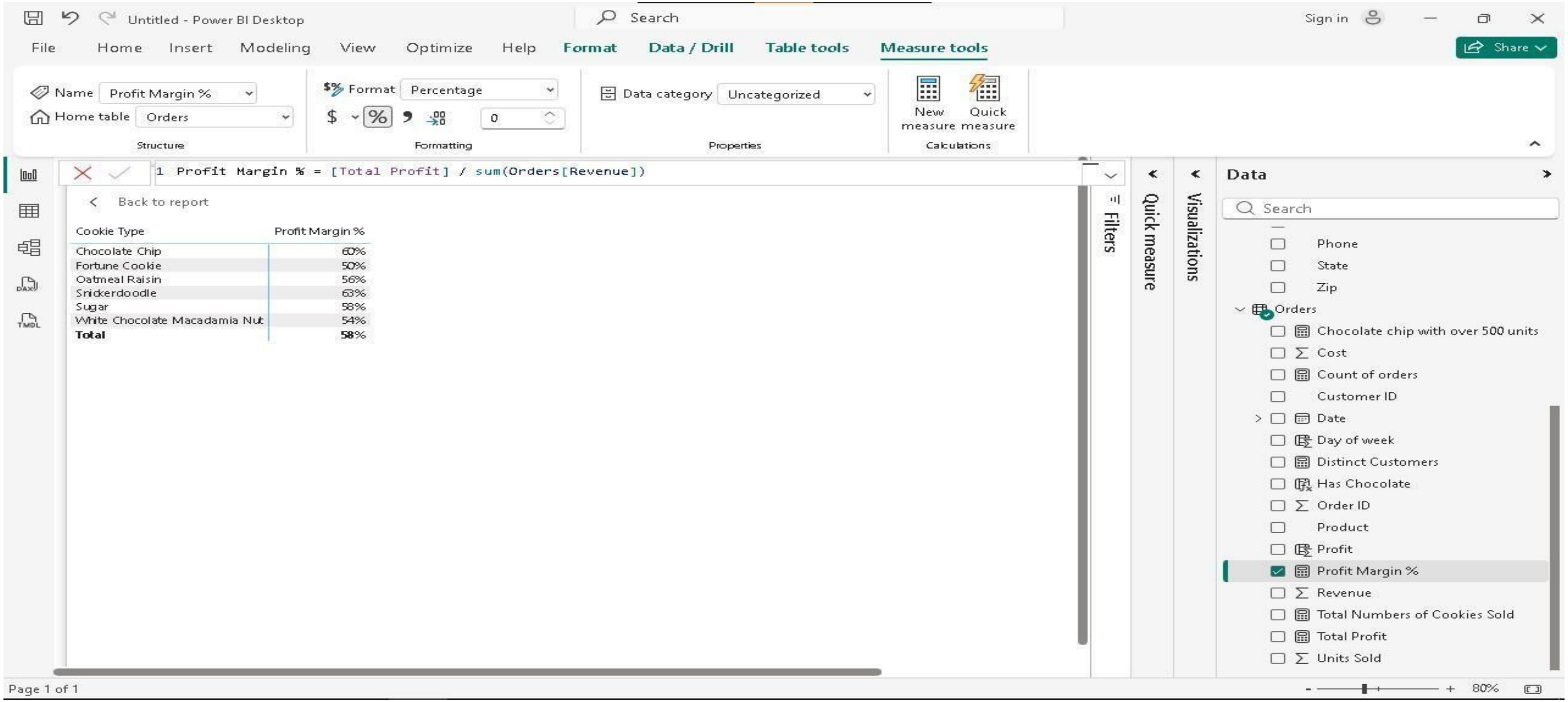


Formula: Profit = Orders[Revenue] - Orders[Cost]

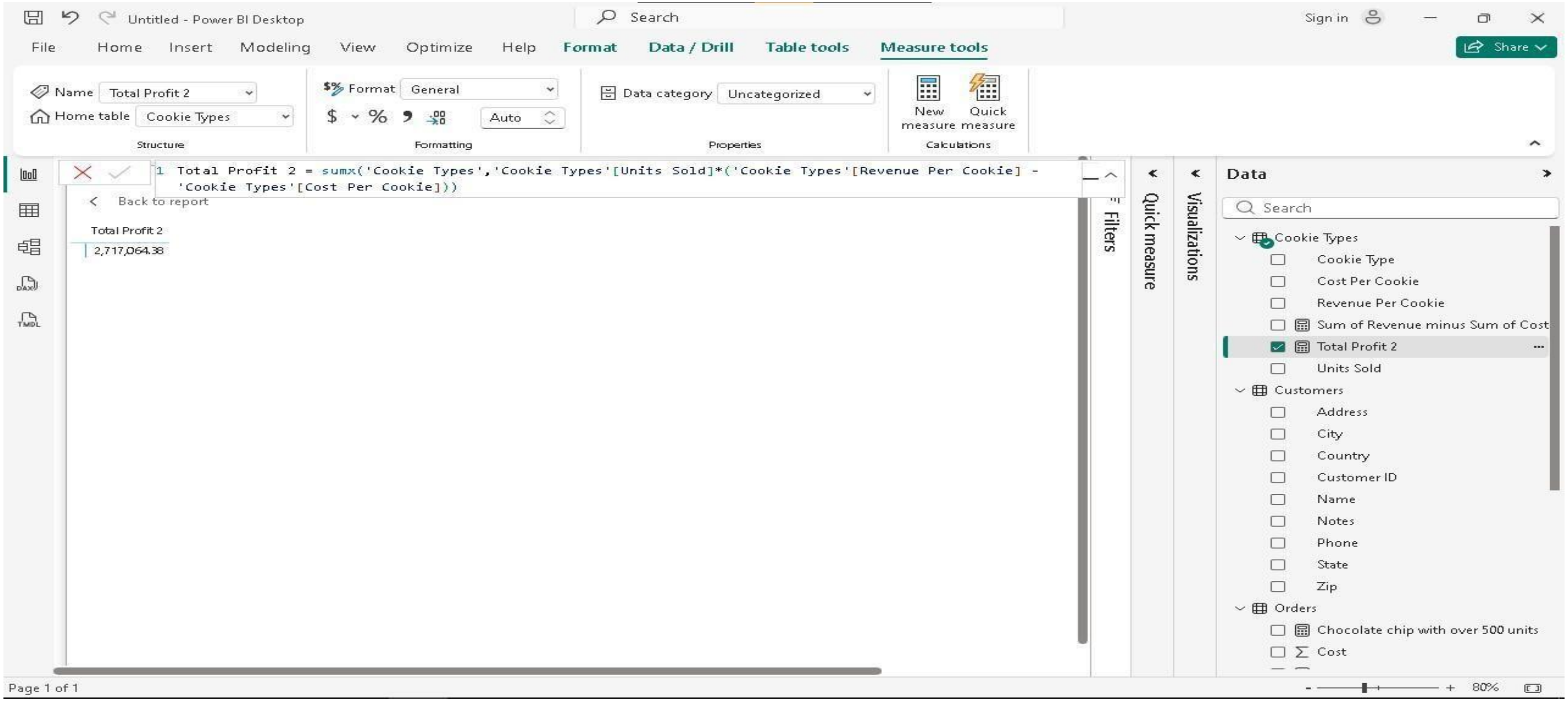


Customer ID	Order ID	Product	Units Sold	Date	Revenue	Cost	Profit	Day of week	Has Chocolate
3	266868	Chocolate Chip	292	Saturday, February 1, 2020	1460	584	876	7	Has Chocolate
3	140794	Chocolate Chip	974	Saturday, February 1, 2020	4870	1948	2922	7	Has Chocolate
3	684759	Chocolate Chip	2518	Monday, June 1, 2020	12590	5036	7554	2	Has Chocolate
3	183251	Chocolate Chip	1513	Tuesday, December 1, 2020	7565	3026	4539	3	Has Chocolate
3	600124	Chocolate Chip	1030	Friday, May 1, 2020	5150	2060	3090	6	Has Chocolate
3	562219	Chocolate Chip	1514	Saturday, February 1, 2020	7570	3028	4542	7	Has Chocolate
3	387444	Chocolate Chip	787	Monday, June 1, 2020	3935	1574	2361	2	Has Chocolate
3	365463	Chocolate Chip	1728	Friday, May 1, 2020	8640	3456	5184	6	Has Chocolate
3	505159	Chocolate Chip	2145	Tuesday, October 1, 2019	10725	4290	6435	3	Has Chocolate
3	300303	Chocolate Chip	1084	Tuesday, December 1, 2020	5420	2168	3252	3	Has Chocolate
3	578401	Chocolate Chip	689	Monday, June 1, 2020	3445	1378	2067	2	Has Chocolate
3	365552	Chocolate Chip	1570	Monday, June 1, 2020	7850	3140	4710	2	Has Chocolate
3	239419	Chocolate Chip	4251	Wednesday, January 1, 2020	21255	8502	12753	4	Has Chocolate
3	549329	Chocolate Chip	2918	Friday, May 1, 2020	14590	5836	8754	6	Has Chocolate
3	288851	Chocolate Chip	2988	Wednesday, July 1, 2020	14940	5976	8964	4	Has Chocolate
3	255145	Chocolate Chip	2074	Tuesday, September 1, 2020	10370	4148	6222	3	Has Chocolate
3	538134	Chocolate Chip	1514	Tuesday, October 1, 2019	7570	3028	4542	3	Has Chocolate
3	817134	Chocolate Chip	274	Tuesday, December 1, 2020	1370	548	822	3	Has Chocolate
3	697568	Chocolate Chip	1138	Tuesday, December 1, 2020	5690	2276	3414	3	Has Chocolate
3	508782	Chocolate Chip	2992	Sunday, March 1, 2020	14960	5984	8976	1	Has Chocolate
3	697895	Chocolate Chip	1228	Tuesday, October 1, 2019	6140	2456	3684	3	Has Chocolate
3	691331	Chocolate Chip	1389	Tuesday, October 1, 2019	6945	2778	4167	3	Has Chocolate
3	568366	Chocolate Chip	1496	Monday, June 1, 2020	7480	2992	4488	2	Has Chocolate
3	176592	Chocolate Chip	2299	Tuesday, October 1, 2019	11495	4598	6897	3	Has Chocolate

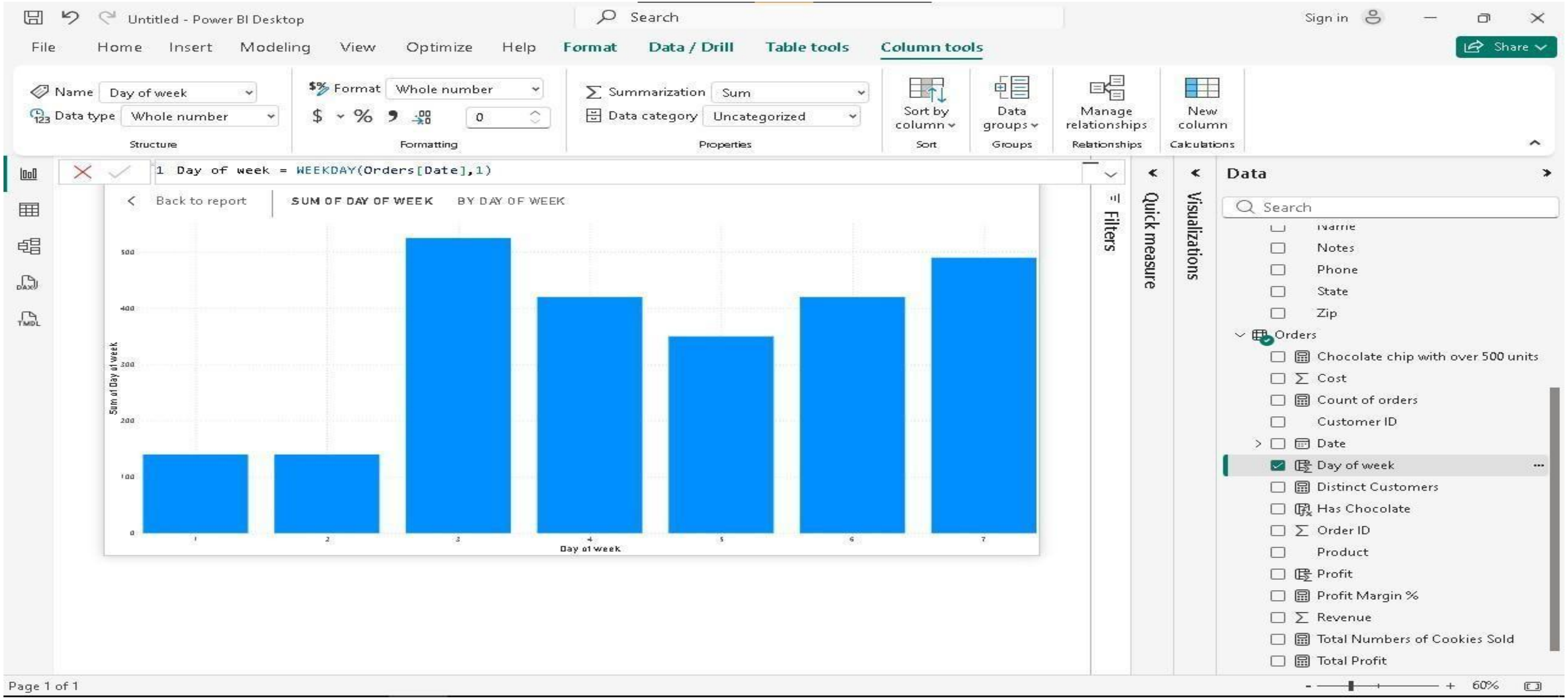
Formula: Profit Margin % = [Total Profit] / sum(Orders[Revenue])



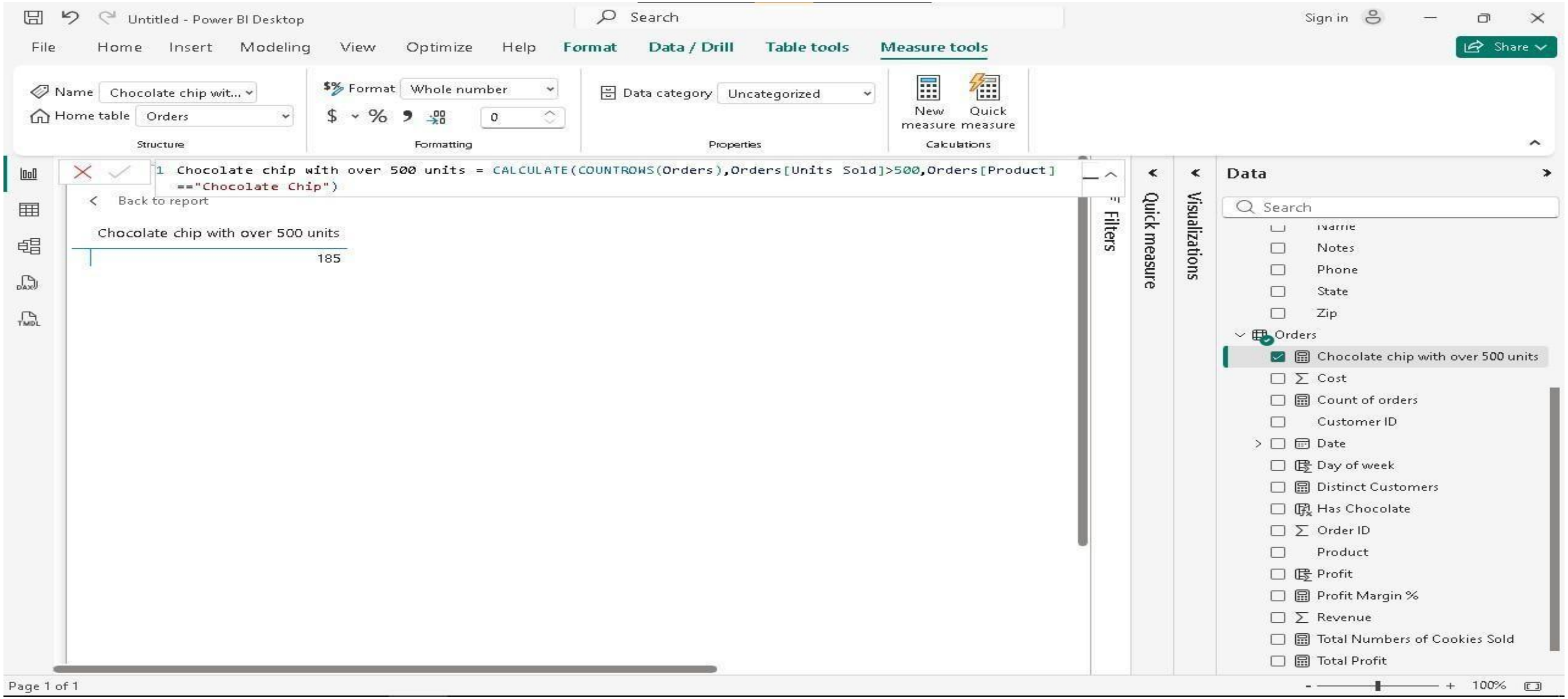
Formula: Total Profit 2 = sumx('Cookie Types','Cookie Types'[Units Sold]*('Cookie Types'[Revenue Per Cookie] - 'Cookie Types'[Cost Per Cookie]))



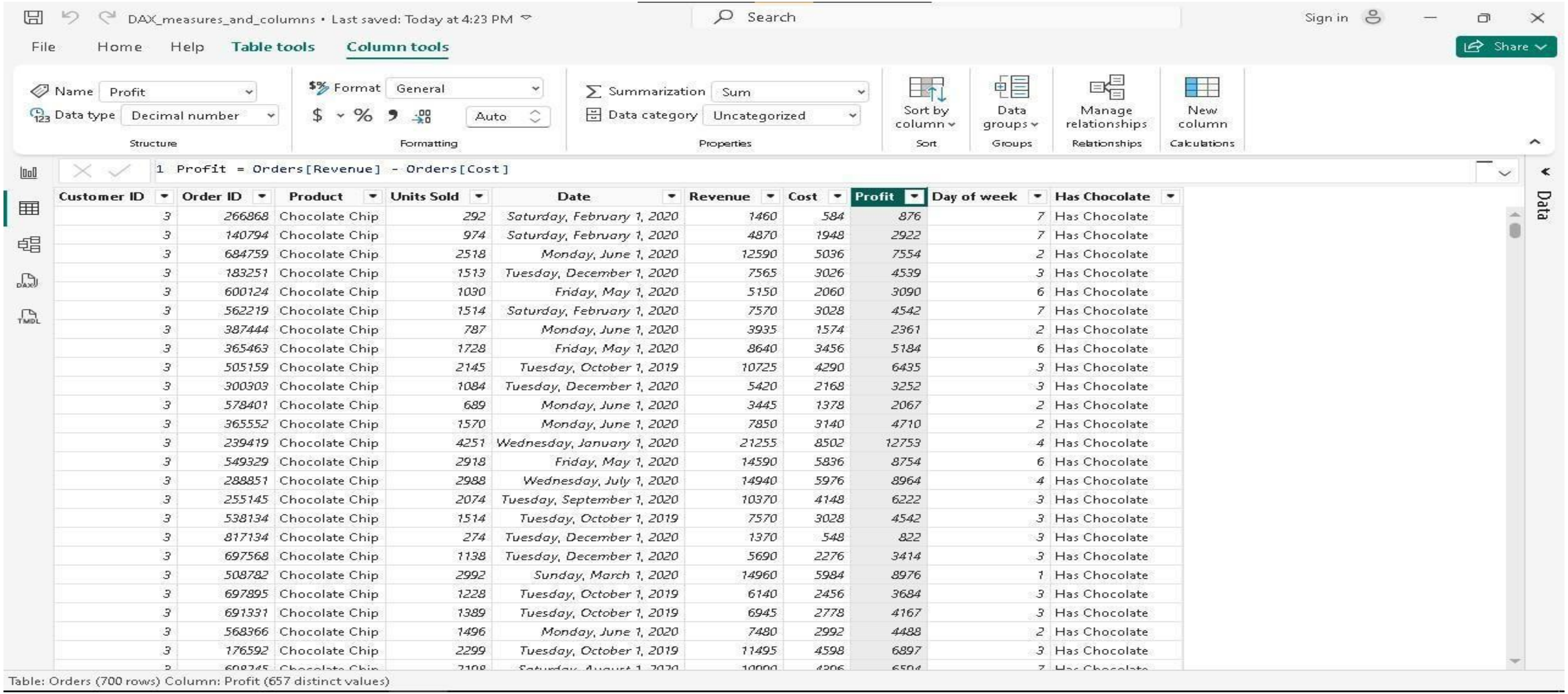
Formula: Day of week = WEEKDAY(Orders[Date],1)



Formula: Chocolate chip with over 500 units = CALCULATE(COUNTROWS(Orders),Orders[Units Sold]>500,Orders[Product]=="Chocolate Chip")



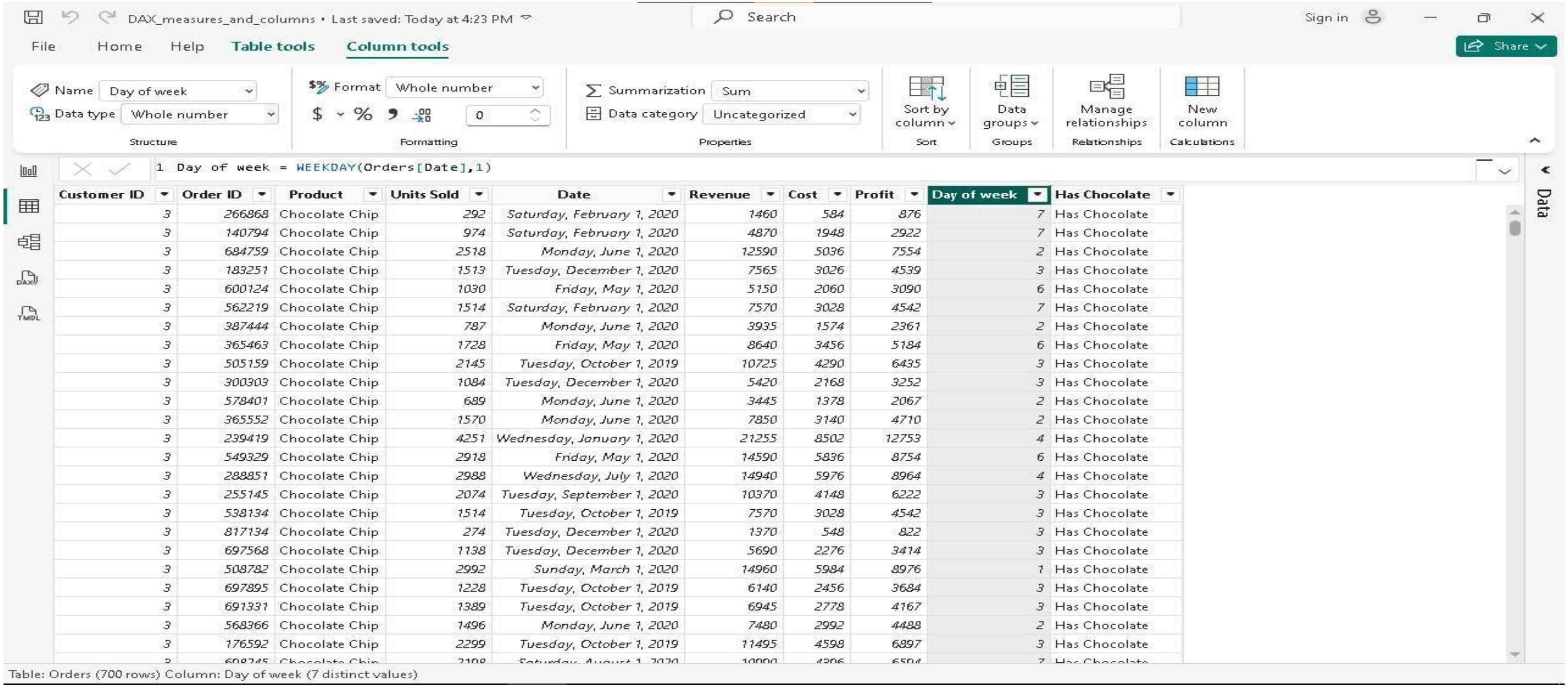
Formula: Profit = Orders[Revenue] - Orders[Cost]



Customer ID	Order ID	Product	Units Sold	Date	Revenue	Cost	Profit	Day of week	Has Chocolate
3	266868	Chocolate Chip	292	Saturday, February 1, 2020	1460	584	876	7	Has Chocolate
3	140794	Chocolate Chip	974	Saturday, February 1, 2020	4870	1948	2922	7	Has Chocolate
3	684759	Chocolate Chip	2518	Monday, June 1, 2020	12590	5036	7554	2	Has Chocolate
3	183251	Chocolate Chip	1513	Tuesday, December 1, 2020	7565	3026	4539	3	Has Chocolate
3	600124	Chocolate Chip	1030	Friday, May 1, 2020	5150	2060	3090	6	Has Chocolate
3	562219	Chocolate Chip	1514	Saturday, February 1, 2020	7570	3028	4542	7	Has Chocolate
3	387444	Chocolate Chip	787	Monday, June 1, 2020	3935	1574	2361	2	Has Chocolate
3	365463	Chocolate Chip	1728	Friday, May 1, 2020	8640	3456	5184	6	Has Chocolate
3	505159	Chocolate Chip	2145	Tuesday, October 1, 2019	10725	4290	6435	3	Has Chocolate
3	300303	Chocolate Chip	1084	Tuesday, December 1, 2020	5420	2168	3252	3	Has Chocolate
3	578401	Chocolate Chip	689	Monday, June 1, 2020	3445	1378	2067	2	Has Chocolate
3	365552	Chocolate Chip	1570	Monday, June 1, 2020	7850	3140	4710	2	Has Chocolate
3	239419	Chocolate Chip	4251	Wednesday, January 1, 2020	21255	8502	12753	4	Has Chocolate
3	549329	Chocolate Chip	2918	Friday, May 1, 2020	14590	5836	8754	6	Has Chocolate
3	288851	Chocolate Chip	2988	Wednesday, July 1, 2020	14940	5976	8964	4	Has Chocolate
3	255145	Chocolate Chip	2074	Tuesday, September 1, 2020	10370	4148	6222	3	Has Chocolate
3	538134	Chocolate Chip	1514	Tuesday, October 1, 2019	7570	3028	4542	3	Has Chocolate
3	817134	Chocolate Chip	274	Tuesday, December 1, 2020	1370	548	822	3	Has Chocolate
3	697568	Chocolate Chip	1138	Tuesday, December 1, 2020	5690	2276	3414	3	Has Chocolate
3	508782	Chocolate Chip	2992	Sunday, March 1, 2020	14960	5984	8976	1	Has Chocolate
3	697895	Chocolate Chip	1228	Tuesday, October 1, 2019	6140	2456	3684	3	Has Chocolate
3	691331	Chocolate Chip	1389	Tuesday, October 1, 2019	6945	2778	4167	3	Has Chocolate
3	568366	Chocolate Chip	1496	Monday, June 1, 2020	7480	2992	4488	2	Has Chocolate
3	176592	Chocolate Chip	2299	Tuesday, October 1, 2019	11495	4598	6897	3	Has Chocolate

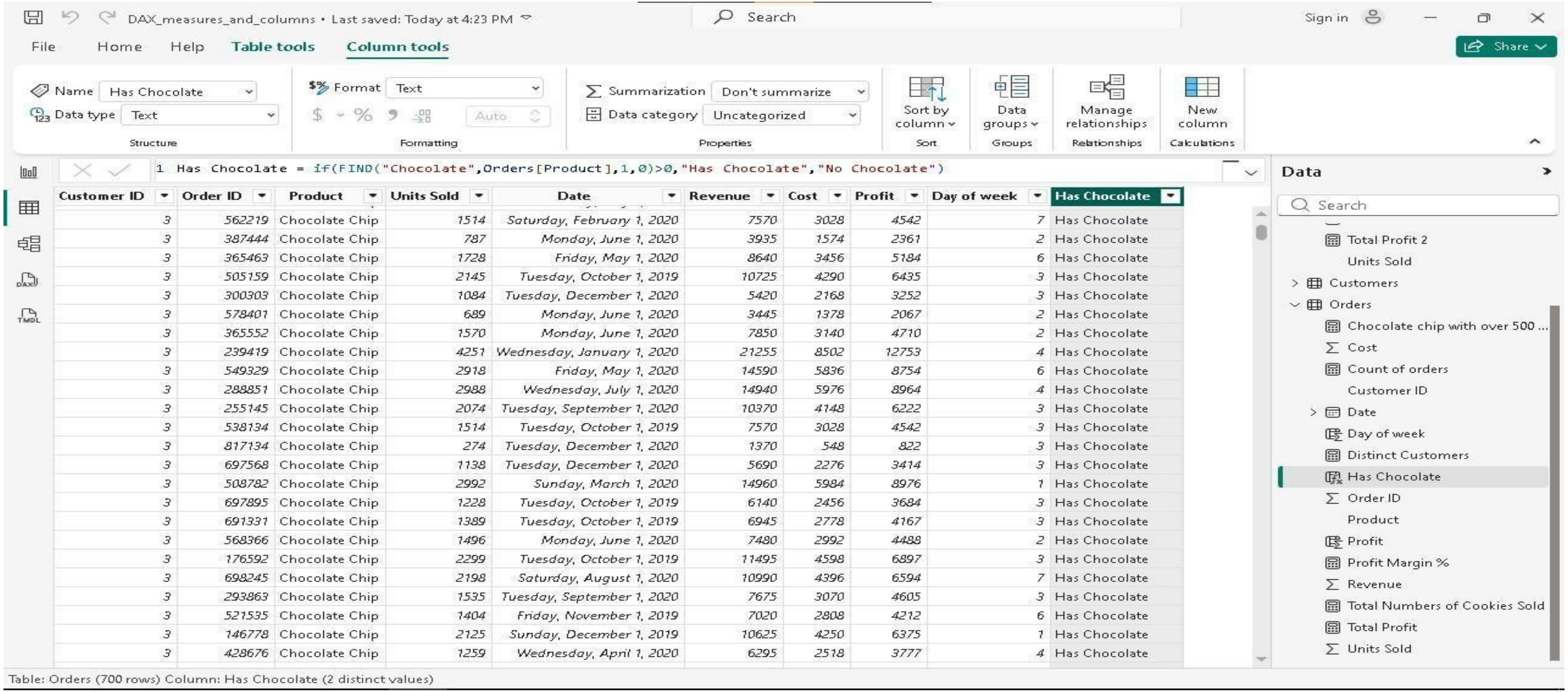
Table: Orders (700 rows) Column: Profit (657 distinct values)

Formula: Day of week = WEEKDAY(Orders[Date],1)



Customer ID	Order ID	Product	Units Sold	Date	Revenue	Cost	Profit	Day of week	Has Chocolate
3	266868	Chocolate Chip	292	Saturday, February 1, 2020	1460	584	876	7	Has Chocolate
3	140794	Chocolate Chip	974	Saturday, February 1, 2020	4870	1948	2922	7	Has Chocolate
3	684759	Chocolate Chip	2518	Monday, June 1, 2020	12590	5036	7554	2	Has Chocolate
3	183251	Chocolate Chip	1513	Tuesday, December 1, 2020	7565	3026	4539	3	Has Chocolate
3	600124	Chocolate Chip	1030	Friday, May 1, 2020	5150	2060	3090	6	Has Chocolate
3	562219	Chocolate Chip	1514	Saturday, February 1, 2020	7570	3028	4542	7	Has Chocolate
3	387444	Chocolate Chip	787	Monday, June 1, 2020	3935	1574	2361	2	Has Chocolate
3	365463	Chocolate Chip	1728	Friday, May 1, 2020	8640	3456	5184	6	Has Chocolate
3	505159	Chocolate Chip	2145	Tuesday, October 1, 2019	10725	4290	6435	3	Has Chocolate
3	300303	Chocolate Chip	1084	Tuesday, December 1, 2020	5420	2168	3252	3	Has Chocolate
3	578401	Chocolate Chip	689	Monday, June 1, 2020	3445	1378	2067	2	Has Chocolate
3	365552	Chocolate Chip	1570	Monday, June 1, 2020	7850	3140	4710	2	Has Chocolate
3	239419	Chocolate Chip	4251	Wednesday, January 1, 2020	21255	8502	12753	4	Has Chocolate
3	549329	Chocolate Chip	2918	Friday, May 1, 2020	14590	5836	8754	6	Has Chocolate
3	288851	Chocolate Chip	2988	Wednesday, July 1, 2020	14940	5976	8964	4	Has Chocolate
3	255145	Chocolate Chip	2074	Tuesday, September 1, 2020	10370	4148	6222	3	Has Chocolate
3	538134	Chocolate Chip	1514	Tuesday, October 1, 2019	7570	3028	4542	3	Has Chocolate
3	817134	Chocolate Chip	274	Tuesday, December 1, 2020	1370	548	822	3	Has Chocolate
3	697568	Chocolate Chip	1138	Tuesday, December 1, 2020	5690	2276	3414	3	Has Chocolate
3	508782	Chocolate Chip	2992	Sunday, March 1, 2020	14960	5984	8976	1	Has Chocolate
3	697895	Chocolate Chip	1228	Tuesday, October 1, 2019	6140	2456	3684	3	Has Chocolate
3	691331	Chocolate Chip	1389	Tuesday, October 1, 2019	6945	2778	4167	3	Has Chocolate
3	568366	Chocolate Chip	1496	Monday, June 1, 2020	7480	2992	4488	2	Has Chocolate
3	176592	Chocolate Chip	2299	Tuesday, October 1, 2019	11495	4598	6897	3	Has Chocolate

Formula: Has Chocolate = if(FIND("Chocolate",Orders[Product],1,0)>0,"Has Chocolate","No Chocolate")



Customer ID	Order ID	Product	Units Sold	Date	Revenue	Cost	Profit	Day of week	Has Chocolate
3	562219	Chocolate Chip	1514	Saturday, February 1, 2020	7570	3028	4542	7	Has Chocolate
3	387444	Chocolate Chip	787	Monday, June 1, 2020	3935	1574	2361	2	Has Chocolate
3	365463	Chocolate Chip	1728	Friday, May 1, 2020	8640	3456	5184	6	Has Chocolate
3	505159	Chocolate Chip	2145	Tuesday, October 1, 2019	10725	4290	6435	3	Has Chocolate
3	300303	Chocolate Chip	1084	Tuesday, December 1, 2020	5420	2168	3252	3	Has Chocolate
3	578401	Chocolate Chip	689	Monday, June 1, 2020	3445	1378	2067	2	Has Chocolate
3	365552	Chocolate Chip	1570	Monday, June 1, 2020	7850	3140	4710	2	Has Chocolate
3	239419	Chocolate Chip	4251	Wednesday, January 1, 2020	21255	8502	12753	4	Has Chocolate
3	549329	Chocolate Chip	2918	Friday, May 1, 2020	14590	5836	8754	6	Has Chocolate
3	288851	Chocolate Chip	2988	Wednesday, July 1, 2020	14940	5976	8964	4	Has Chocolate
3	255145	Chocolate Chip	2074	Tuesday, September 1, 2020	10370	4148	6222	3	Has Chocolate
3	538134	Chocolate Chip	1514	Tuesday, October 1, 2019	7570	3028	4542	3	Has Chocolate
3	817134	Chocolate Chip	274	Tuesday, December 1, 2020	1370	548	822	3	Has Chocolate
3	697568	Chocolate Chip	1138	Tuesday, December 1, 2020	5690	2276	3414	3	Has Chocolate
3	508782	Chocolate Chip	2992	Sunday, March 1, 2020	14960	5984	8976	1	Has Chocolate
3	697895	Chocolate Chip	1228	Tuesday, October 1, 2019	6140	2456	3684	3	Has Chocolate
3	691331	Chocolate Chip	1389	Tuesday, October 1, 2019	6945	2778	4167	3	Has Chocolate
3	568366	Chocolate Chip	1496	Monday, June 1, 2020	7480	2992	4488	2	Has Chocolate
3	176592	Chocolate Chip	2299	Tuesday, October 1, 2019	11495	4598	6897	3	Has Chocolate
3	698245	Chocolate Chip	2198	Saturday, August 1, 2020	10990	4396	6594	7	Has Chocolate
3	293863	Chocolate Chip	1535	Tuesday, September 1, 2020	7675	3070	4605	3	Has Chocolate
3	521535	Chocolate Chip	1404	Friday, November 1, 2019	7020	2808	4212	6	Has Chocolate
3	146778	Chocolate Chip	2125	Sunday, December 1, 2019	10625	4250	6375	1	Has Chocolate
3	428676	Chocolate Chip	1259	Wednesday, April 1, 2020	6295	2518	3777	4	Has Chocolate

Guided Questions:

I. Foundational Concepts

1. Defining DAX: In your own words, how would you explain the difference between a standard Excel formula and a Data Analysis Expression (DAX) based on the tutorial?

Answer: A standard Excel formula works on a fixed cell or range it's static and tied to a specific location in a spreadsheet. DAX, on the other hand, is designed to work with relational data models and entire columns or tables. Instead of referencing "cell B2," DAX references "the Sales Amount column in the Sales table." It's also dynamic it responds to filters, slicers, and visuals automatically, something Excel formulas can't do natively in the same way.

Guided Questions:

I. Foundational Concepts

2. The "Why" of DAX: Why can’t we just use the raw numbers from our imported dataset to create complex reports? What is the specific "gap" that DAX fills in Power BI?

Answer: Raw numbers from an imported dataset are just stored values they can't adapt to context. The gap DAX fills is dynamic calculation. For example, raw data might have individual sales rows, but to get "Total Sales only for a specific region when a slicer is clicked," you need DAX. It allows you to create calculations that respond to user interaction and filter context in real time, which raw data simply cannot do.

II. Calculated Columns vs. Measures

3. The Storage Difference: The video demonstrates creating both Calculated Columns and Measures. Which one actually increases the file size of your Power BI model, and why is this an important consideration for large datasets?

Answer: Calculated Columns increase the file size because they are computed row-by-row at data refresh time and stored physically inside the data model. Every new row gets a new computed value saved to memory. Measures, by contrast, are calculated on the fly only when a visual needs them nothing is stored. For large datasets with millions of rows, using Calculated Columns unnecessarily can dramatically bloat your .pbix file.

Guided Questions:

II. Calculated Columns vs. Measures

4. Context Clues: * When you created a Calculated Column, the calculation happened row-by-row.

○ When you created a Measure, the result changed depending on which filters or visuals you clicked.

○ Question: Explain how "Filter Context" affects the result of a Measure in your final dashboard.

Answer:Filter Context is the set of filters currently active in a report from slicers, visual filters, row/column headers in a matrix, or cross-filtering from other visuals. A Measure's result automatically adjusts based on whatever Filter Context surrounds it. For example, a Total Sales measure will return the overall total on an empty canvas, but if you place it inside a chart broken down by Region, it will return the sales per region the same formula, different results, because the Filter Context changed. This makes Measures incredibly powerful and flexible.

Guided Questions:

III. Function Application & Syntax

5. The RELATED Function: In the tutorial, the RELATED function is used. Why is this function necessary when working with multiple tables, and what must be established in the Model View for this function to work correctly?

Answer:RELATED is necessary when you need to pull a value from a different table into a Calculated Column. Since Power BI works with relational tables (not one big flat sheet), columns from other tables aren't directly accessible in a formula. For RELATED to work, a relationship must be established in Model View between the two tables specifically, a one-to-many relationship where the table you're pulling from is on the "one" side. Without that relationship, RELATED has no path to follow and will throw an error.

Guided Questions:

III. Function Application & Syntax

6. Deconstructing the Formula: Look at the following formula from the video:

Component	Value
Measure Name	Total Sales
Function	SUM()
Table	Sales
Column	Sales Amount

Total Sales = SUM(Sales[Sales Amount]) Identify the Measure Name, the Function, the Table, and the Column. Why is it "best practice" to include the table name in the reference?

Answer: Total Sales = SUM(Sales[Sales Amount])

Guided Questions:

III. Function Application & Syntax

6. Deconstructing the Formula: Look at the following formula from the video:

Total Sales = SUM(Sales[Sales Amount]) Identify the Measure Name, the Function, the Table, and the Column. Why is it "best practice" to include the table name in the reference?

Answer: Including the table name (Sales[Sales Amount] instead of just [Sales Amount]) is best practice because it removes ambiguity if multiple tables have a column named "Sales Amount," DAX knows exactly which one you mean. It also makes formulas easier to read, debug, and maintain.

Guided Questions:

IV. Analysis & Troubleshooting

7. Data Interpretation: Look at the visual you created using your DAX measures. If the "Total Profit" measure shows a negative number for a specific category, does that mean the DAX formula is "broken," or is it telling you something about the business data? How can you verify which it is?

Answer: A negative "Total Profit" does not mean the formula is broken it's most likely telling you something real about the business: that category is operating at a loss (costs exceed revenue). To verify, you should:

- Check the underlying data for that category (are the cost values higher than the sales values?)
- Simplify the measure temporarily to display just Total Revenue and Total Cost separately to see both sides
- Cross-check with the source dataset to confirm the raw numbers match

Guided Questions:

IV. Analysis & Troubleshooting

8. Optimization: If you were tasked with calculating Profit Margin Percentage, would it be more efficient to create a Calculated Column for every row or a single Measure? Justify your answer based on what you learned about DAX efficiency.

Answer: A single Measure is far more efficient. Here's why:

- A Calculated Column would compute and store the profit margin for every single row in the table, consuming memory and increasing file size even for rows you may never display.
- A Measure calculates the profit margin only when needed, and it automatically adapts to whatever filter context is active (e.g., per product, per region, per month).
- Profit Margin is an aggregated insight, so it makes far more sense as a Measure: Profit Margin % = DIVIDE([Total Profit], [Total Sales], 0) clean, dynamic, and storage-efficient.